One Fusion Does Not Fit All: Kernel-Aware Multimodal Expert Selection for HLS QoR Prediction

***Abstract*—High-Level Synthesis (HLS) design space exploration (DSE) demands fast and accurate Quality-of-Results (QoR) prediction across diverse pragma configurations to avoid expensive repeated synthesis. Existing learning-based predictors mostly rely on fixed combinations of control/data-flow graphs (CDFGs) and textual features, failing to fully capture source-level syntax such as pragma–loop associations and loop nesting, and offering limited generalization across kernels and QoR metrics. To address these limitations, we propose a Multimodal Feature Fusion (MFF) framework that integrates CDFG, abstract syntax tree (AST), and source-code Text to build comprehensive program representations for HLS QoR prediction. Second, we train seven modality specific fusion experts, one for each non empty subset of CDFG, AST, Text, each performing multi output regression over five QoR metrics (Latency, LUT, FF, DSP, and BRAM). Through extensive experiments, we discover for the first time that no single fixed modality combination dominates across all kernels and QoR metrics. Motivated by this finding, we propose a meta-learning based kernel-aware selector (KAS) that dynamically chooses the most suitable pretrained expert for each unseen kernel and QoR metric at inference time. On unseen test kernels, trimodal fusion based on MFF (*AST+CDFG+Text*) reduces macro-averaged *All* RMSE by 13.1%–52.9% over SOTA predictors (GNN-DSE, HGBO, Ironman-Pro, and MPM). MFF-KAS further reduces *All* RMSE by 30.5% over trimodal fusion and by 39.6%–67.3% over SOTA baselines, with stronger cross-kernel robustness. The code and models are available at https://anonymous.4open.science/r/MFF-KAS-EFE0.**

***Index Terms*—High-level synthesis, Graph neural networks, Abstract syntax tree, Multimodal fusion, Meta-learning**

I. Introduction

High-Level Synthesis (HLS) automatically transforms high-level programming languages such as C/C++ into Hardware Description Languages (HDLs), significantly lowering the design barrier for FPGA development. In HLS Design Space Exploration (DSE), the combinations of pragmas and compilation options grow exponentially. Invoking synthesis and implementation for every configuration therefore incurs prohibitively high costs [1]. As a result, rapidly and accurately estimating Quality of Results (QoR), including latency and resource utilization metrics such as LUT, FF, DSP, and BRAM, before synthesis has become a critical step for filtering candidate designs and approaching Pareto-optimal solutions [2].

In recent years, machine learning (ML)-based surrogate models have gradually replaced traditional analytical estimation methods [3]. Among them, approaches that use the Control Data Flow Graph (CDFG) as input and adopt Graph Neural Networks (GNNs) to learn the mapping between program structures and QoR have become the dominant paradigm [4]. The upper bound of QoR prediction performance largely depends on the quality of the input representation. When pragma parameters or their attachment to specific loops and statements are not preserved in the model input, even stronger GNN architectures struggle to consistently capture their impact on performance and resource metrics.

Along the graph-based QoR prediction line, GNN-DSE [4] encodes HLS programs as control–dataflow graphs (CDFGs) and employs GNNs to learn operator dependencies and dataflow relationships, enabling QoR estimation without repeatedly invoking synthesis. HGBO-DSE [5] introduces a hierarchical graph predictor (HGP) that improves prediction and search efficiency in multi-objective scenarios through multi-granularity CDFGs. IronMan-Pro [6] further strengthens GNN architectures and message passing over program graphs exported from HLS tools. Although effective for modeling dependencies and dataflow, these methods adopt a CDFG-centric, graph-only representation and therefore offer limited access to source-level syntactic detail.

To compensate for missing program semantics, subsequent work fuses graphs with textual programs. Wang and Carrion-Schafer [7] show that syntax-oriented program features, including abstract-syntax-tree descriptors, combined with conventional machine-learning models can accelerate FPGA HLS design-space exploration by reusing knowledge from previously explored designs. ProgSG [8] aligns source-code tokens with CDFG nodes for cross-modality representation learning. MPM [9] couples CDFG encoders with pretrained language-model embeddings in a CDFG+Text fusion pipeline, achieving strong accuracy on MachSuite and PolyBench. Nevertheless, such bimodal schemes still omit explicit abstract syntax trees (ASTs) and therefore under-represent pragma parameters, their attachment to specific loops or statements, and nested loop structure. Moreover, their modality combination and network topology are fixed at training time, so the same model is applied at inference to every kernel and every QoR metrics (e.g., Latency, LUT, FF, and DSP). even though the best fusion strategy varies across kernels and metrics.

In summary, the field has progressed from CDFG-only predictors to CDFG+Text fusion based predictors, yet a systematic solution is still lacking for syntax-aware multimodal modeling and for adapting representations across heterogeneous kernels and QoR metrics. Motivated by the above limitations, especially the incomplete capture of pragma–loop structure

TABLE I: Comparison of three feature representations for HLS QoR prediction.

Feature	Form	Information Content	Advantages	Limitations
CDFG	Graph	Basic blocks, operations, and data/control dependencies	Aligned with hardware timing and datapaths	Easily loses data types and exact pragma values
Text	Sequence	Entire source and pragma text	Complete textual content	Lacks hierarchical structure
AST	Tree	Functions, branches, loops, expressions, and pragmas	Preserves syntax and high-level semantics	Lacks explicit dataflow and control flow

in both CDFG-only and CDFG+Text pipelines, we introduce the abstract syntax tree as a complementary third representation [10]. Parsed directly from C/C++ with compilers, the AST preserves functions, loops, conditions, expressions, and pragmas, and explicitly records which pragmas attach to which statements or loops. It thus complements CDFG views that emphasize data/control flow and Text embeddings that lack explicit hierarchical syntax.

To make this distinction concrete, consider `#pragma HLS UNROLL factor=4` as an example. This directive appears in Text representations merely as a sequence of discrete tokens, making it difficult to establish structural associations with the target loop body. In CDFG, it is typically compressed into an auxiliary attribute of a loop node, which inadequately reflects the changes in parallelism and dataflow reorganization introduced by variations in the unroll factor. In contrast, within the AST, the pragma can be represented as an independent node attached directly to the corresponding `ForStmt`, where both the unroll factor and the loop-nesting hierarchy are explicitly visible.

These comparisons indicate that, for HLS QoR prediction tasks requiring the preservation of both optimization directive parameters and their syntactic context, incorporating AST representations is both necessary and well justified [10]. Table I summarizes the three representations in terms of representation form, information focus, advantages, and limitations.

Building upon this foundation, this paper proposes the Multimodal Feature Fusion method (MFF) as its first core contribution. MFF encodes abstract syntax trees (ASTs) to capture syntactic structure and deeply integrates them with the computational-flow information in CDFGs and the semantic context in source-code Text. Through this multimodal fusion, MFF explicitly models loop nesting, memory layouts, and pragma–loop/statement associations, yielding a more comprehensive program representation for QoR prediction.

However, richer multimodal representations do not always guarantee uniformly optimal QoR prediction. We systematically evaluate all seven non-empty subsets of {CDFG, AST, Text}: {CDFG+AST+Text}, {CDFG+AST}, {AST+Text}, {CDFG+Text}, CDFG-only, AST-only, and Text-only. Experiments on MachSuite and PolyBench show that the best subset varies across kernels and QoR metrics (Latency, LUT, FF, DSP, and BRAM); no single fusion strategy dominates all cases, consistent with the no-free-lunch theorem (NFL) [11].

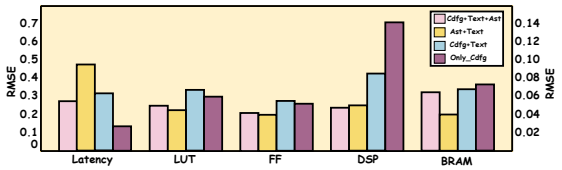


Fig. 1. RMSE of four representative fusion schemes on the *jacobi-2d* kernel across five QoR metrics. Latency is plotted against the **left** y-axis; LUT, FF, DSP, and BRAM RMSE are plotted against the **right** y-axis.

Inspired by algorithm selection [12], we propose a kernel-aware selector (KAS) that learns, across diverse kernels and metrics, to select for each (kernel, metric) pair the most suitable of seven pretrained fusion experts. At inference time, KAS reads multimodal features from CDFG, AST, and Text together with the QoR metric and selects the corresponding expert, avoiding a globally fixed fusion policy.

The main contributions of this work are summarized as follows:

- We introduce an AST-based GNN encoding for HLS QoR prediction, which preserves pragma–loop associations and syntactic hierarchy that are difficult to retain in CDFG-only or sequence-only representations.
- We propose MFF, a Multimodal Feature Fusion framework that integrates CDFG, AST, and source-code Text through cross-attention-based fusion, and systematically evaluates all seven non-empty subsets of {CDFG, AST, Text}.
- Through extensive experiments, we show that no single modality fusion is uniformly optimal across kernels and QoR metrics; accordingly, we design meta-learning based kernel aware selector to adaptively select among seven pretrained experts per kernel and metric, achieving improved prediction accuracy and cross-kernel generalization over SOTA models.

II. PRELIMINARIES

This section presents the unified problem definitions and notations used throughout the remainder of the paper. Section A introduces the multimodal QoR prediction task underlying MFF, while Section B describes the data annotation and expert selection process employed by KAS.

A. Multimodal Graph Learning for HLS QoR Prediction

Consider a hardware kernel k and a configuration c within its design space, where the configuration is jointly determined by the source code and pragma settings. After HLS synthesis and implementation, each (k, c) pair produces a QoR vector $y(k, c)$, which includes performance-related metrics (Latency) as well as resource-related components such as LUT, FF, DSP, and BRAM utilization. Since design space exploration requires evaluating QoR over a large number of configurations, this work aims to learn a pre-synthesis surrogate mapping for QoR prediction [4]. Given the ground-truth QoR vector $y(k, c)$ obtained from HLS synthesis, we learn a surrogate f such that

$$\hat{y}(k, c) = f(k, c), \tag{1}$$

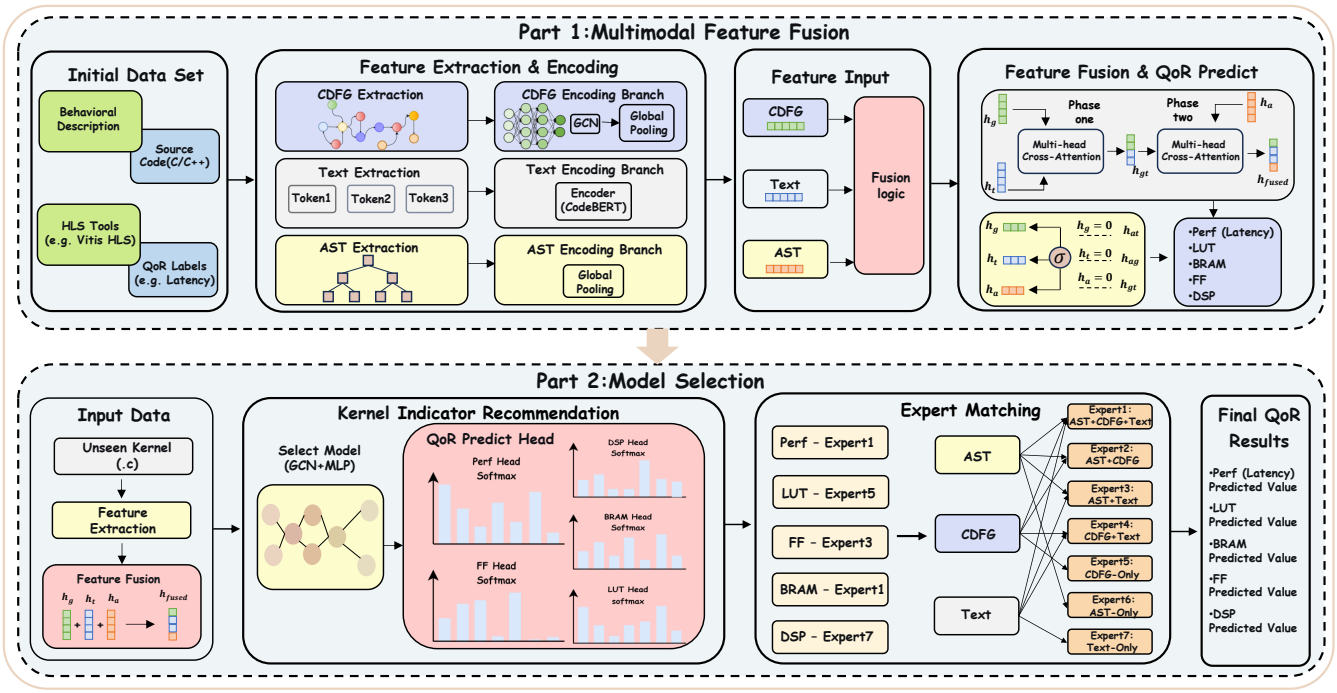


Fig. 2. Overall architecture of MFF-KAS. **Part 1:** CDFG, Text, and AST are encoded (GCN, CodeBERT, and AST-GNN) and fused in MFF via cross-attention for per-metric QoR regression. **Part 2:** KAS selects, for each unseen kernel and QoR metric, one of seven experts pretrained on the seven modality subsets of $\{\mathrm{CDFG}, \mathrm{AST}, \mathrm{Text}\}$.

where $\hat{y}(k,c)$ is trained to approximate $y(k,c)$.

For the same (k,c) pair, three types of representations are constructed (as summarized in Table I). The CDFG is constructed from LLVM IR via program-analysis tools (e.g., ProGraML) [13] and captures data paths as well as control dependencies. The Text representation is obtained by encoding the source code with a pretrained code language model [14], [15] followed by pooling, thereby providing global semantic information. The AST is generated using Tree-sitter parsing, preserving the syntactic hierarchy of loops, branches, expressions, and pragmas. After converting the CDFG and AST into graph structures, graph-level features are extracted using graph neural networks [16], while the Text representation participates in the subsequent fusion process in vector form. Since these three modalities emphasize different aspects of information, they can be flexibly combined. Considering all non-empty subsets of CDFG, AST, Text, a total of seven modality combinations are obtained: CDFG+AST+Text, CDFG+AST, AST+Text, CDFG+Text, CDFG-only, AST-only, and Text-only. Each modality combination corresponds to an expert regressor. For a modality subset $\mathcal{M}$ and a QoR metric t, each expert predicts

$$\hat{y}^{(t)}_{\mathcal{M}}(k,c) = f^{(t)}_{\mathcal{M}}(\mathbf{x}_{\mathcal{M}}(k,c)), \tag{2}$$

where $\mathbf{x}_{\mathcal{M}}(k,c)$ denotes the multimodal input constructed from (k,c) under $\mathcal{M}$. When $|\mathcal{M}| = 1$, $f^{(t)}_{\mathcal{M}}$ is a single-modality regressor; when $|\mathcal{M}| \geq 2$, MFF first fuses the graph- and sequence-level representations of CDFG, AST, and Text into a unified embedding, and a metric-specific head then outputs $\hat{y}^{(t)}$. The fusion applies projection and cross-attention [17] across heterogeneous modalities, with a gated residual to balance the graph features and the fused representation.

B. Multimodal Data and Kernel-Aware Expert Selection

We consider kernels from MachSuite [18] and PolyBench [19]. For each kernel k and configuration c, we construct three modality-specific inputs from the same source program and pragma settings: a CDFG graph, a Text embedding of source code, and an AST parse tree. Thus, every sample (k,c) is associated with a consistent triple of CDFG, Text, and AST features for fusion and expert evaluation.

Each regression expert is trained separately with synthesized QoR values as supervision. After all seven experts are pretrained, we evaluate them on the validation split: given a fixed (k,c) and QoR metric t, we run every expert and record prediction error $e^{(t)}_j(k,c)$; smaller error indicates a better expert. The best expert index is

$$e^*(k,c,t) = \arg\min_{j\in\{0,\ldots,6\}} e^{(t)}_j(k,c). \tag{3}$$

Labels derived from e^* are stored offline and used to train KAS.

As illustrated in Fig. 1, e^* changes across kernels and metrics t, so a single fixed modality combination is inappropriate at inference. We treat the seven experts as a set $\mathcal{E}$. KAS reads the same CDFG, AST, and Text representations as MFF, together with QoR metric t, and selects one expert $\hat{e}(k,t) \in \mathcal{E}$ for each kernel–metric pair (k,t). The selected expert then produces the final prediction for every configuration of kernel k:

$$\hat{e}(k,t) = \mathcal{S}(k,t), \qquad \hat{y}^{(t)} = f^{(t)}_{\mathcal{M}(\hat{e})}\big(\mathbf{x}_{\mathcal{M}(\hat{e})}(k,c)\big), \tag{4}$$

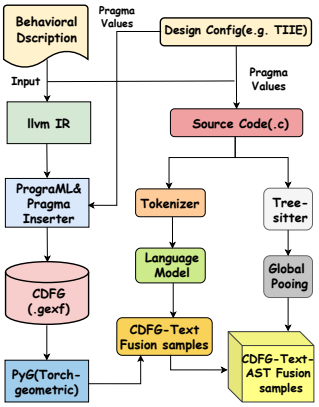


Fig. 3. Construction of aligned CDFG, Text, and AST features for each (k, c).

where $\mathcal{S}$ denotes the selector and $\mathcal{M}(\hat{e})$ is the modality subset of expert $\hat{e}$. The selector maintains a separate scoring head for each QoR metric over $\mathcal{E}$.

III. Methodology

Fig. 2 illustrates the proposed MFF-KAS framework. Part 1 builds seven modality-specific QoR regression experts: multimodal inputs are extracted from each design configuration, fused when more than one modality is active, and mapped to Latency, LUT, FF, DSP, and BRAM through five metric-specific regression heads that share one encoder–fusion backbone per expert. Part 2 trains a kernel-aware selector with reinforcement learning: for an unseen kernel and a QoR metric, KAS recommends one pretrained expert and returns the prediction from the corresponding head. Symbols and Eqs. (1)–(4) follow Section II.

A. Overview

We study HLS QoR prediction on benchmarks such as MachSuite [18] and PolyBench [19]. Each kernel k denotes a synthesizable design (e.g., a loop nest or accelerator), and each configuration c corresponds to a specific pragma and tool setting [20] that yields a distinct implementation. A design configuration (k, c) is therefore identified by source code, pragma annotations, and post-synthesis QoR labels $y(k, c)$ obtained from FPGA HLS tools (e.g., Vitis HLS) [21].

For every (k, c), three parallel representations are derived from the same C/C++ source so that the seven QoR experts and KAS share a consistent input interface. The experts differ only in the enabled modality subset $\mathcal{M}$ in Eq. (2) and are trained offline on the training split with synthesized QoR labels. Best expert indices e^* for KAS supervision are defined in Sec. II-B.

B. Multimodal Feature Extraction

Fig. 3 summarizes the data pipeline. Each modality is encoded into a fixed-dimensional vector or graph embedding before MFF fusion or KAS-based expert selection.

CDFG. Source code is first compiled to LLVM IR [22] by a compiler front end. Program-analysis tools (e.g., ProGraML) [13] construct a CDFG whose nodes denote LLVM IR instructions and whose edges represent data and control dependences. Pragma-related information is attached during graph construction [4] so that design parameters are reflected in the graph topology and node attributes. For learning, the CDFG is represented as a PyTorch Geometric (PyG) graph [23] and encoded by a graph neural network (GNN) with global pooling, yielding a graph-level embedding $\mathbf{h}^G$.

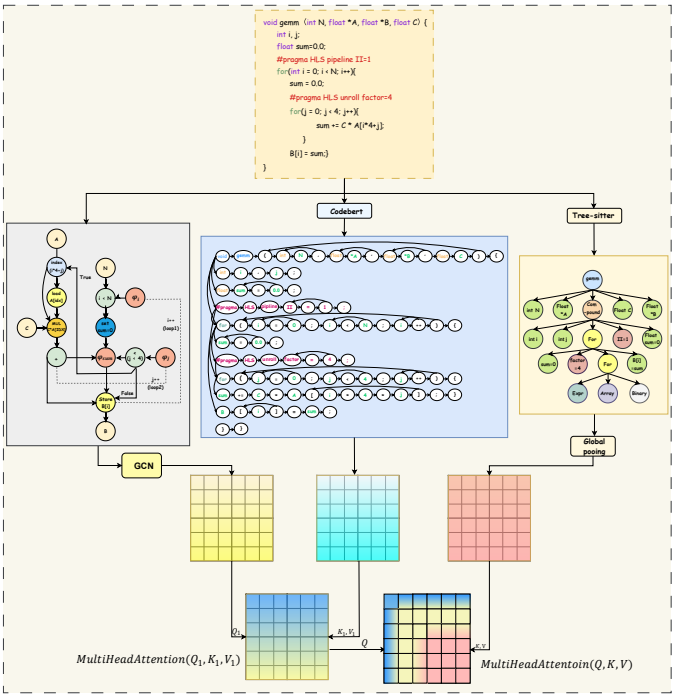


Fig. 4. Two-stage multi-head cross-attention fusion (MFF) for multimodal QoR regression.

Text. The concatenation of source and pragma text is tokenized and encoded by a pretrained code language model (e.g., CodeBERT [14]). We take the pooled hidden state over the sequence as $\mathbf{h}^S$, which captures global semantic cues such as algorithm structure and pragma keywords.

AST. An abstract syntax tree is obtained by parsing C/C++ with Tree-sitter [10]. The AST is converted into a graph and encoded by a separate GNN with pooling [16] to obtain $\mathbf{h}^A$, preserving hierarchical syntax of loops, branches, and pragma placement.

The three embeddings are aligned per (k, c) and cached for expert training, KAS training, and inference.

C. Multimodal Feature Fusion (MFF)

Fig. 4 details the MFF module used in Part 1 of Fig. 2. Given $\mathbf{h}^G$, $\mathbf{h}^S$, and $\mathbf{h}^A$ from Sec. III-B, each expert e uses a modality subset $\mathcal{M}_e$; any disabled modality is zeroed before projection so that one fusion stack supports seven input masks. MFF first fuses CDFG with Text and then injects AST in two cross-attention stages.

Stage 1: Fusing CDFG and Text. We fuse graph-level CDFG with sequence-level Text before adding AST. Because $\mathbf{h}^G$, $\mathbf{h}^S$, and $\mathbf{h}^A$ have different native sizes, we linearly project each active embedding to a shared fusion dimension d before cross-attention. $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ denote the query, key, and value

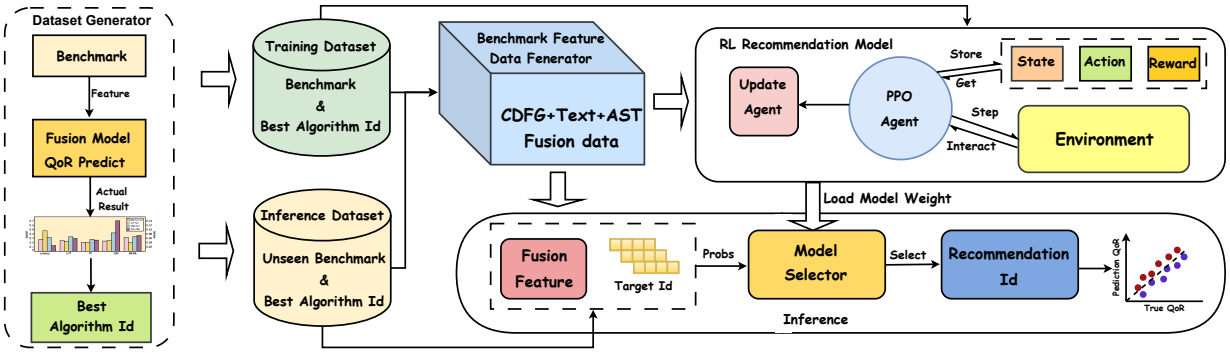


Fig. 5. KAS: label generation from seven experts, multimodal context encoding, PPO-based policy training, and inference-time expert matching.

matrices in cross-attention. For Stage 1, $\mathbf{Q}_1$ is formed from $\mathbf{h}^G$ and $(\mathbf{K}_1, \mathbf{V}_1)$ from $\mathbf{h}^S$. With H heads,

$$\mathbf{h}^{GS} = \mathrm{MHCA}(\mathbf{Q}_1, \mathbf{K}_1, \mathbf{V}_1), \tag{5}$$

where each head uses

$$\mathrm{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}. \tag{6}$$

Here d_k is the per-head key dimension used for scaled dot-product attention; we set $d_k = d/H$, where d is the shared fusion dimension and H is the number of attention heads. A residual connection and layer normalization on the CDFG query branch preserve the graph anchor.

Stage 2: Incorporating AST. Using Eq. (6), $\mathbf{h}^A$ is merged into $\mathbf{h}^{GS}$ with $\mathbf{Q}_2$ from $\mathbf{h}^{GS}$ and $(\mathbf{K}_2, \mathbf{V}_2)$ from $\mathbf{h}^A$, yielding $\mathbf{h}^{\text{fuse}}$ with the same residual pattern.

Gating. Cross-attention fusion can over-emphasize Text/AST context and under-use the CDFG embedding that carries dataflow and control structure. We therefore gate the fused vector with the projected CDFG representation:

$$\begin{aligned} \boldsymbol{\alpha} &= \sigma\big(\mathbf{W}_g[\mathbf{h}^G; \mathbf{h}^{\text{fuse}}]\big)\,, \\ \mathbf{h}^{\text{fuse}'} &= \boldsymbol{\alpha} \odot \mathbf{h}^G + (1-\boldsymbol{\alpha}) \odot \mathbf{h}^{\text{fuse}}. \end{aligned} \tag{7}$$

where $\boldsymbol{\alpha}$ is broadcast over feature dimensions. The gated output $\mathbf{h}^{\text{fuse}'}$ is fed into five metric-specific MLP regression heads—one for each component of $\mathbf{y}(k,c)$ (Latency, LUT, FF, DSP, and BRAM). The heads share the multimodal encoder and fusion trunk but use independent parameters and are optimized jointly in a single forward pass. Each modality-specific expert is thus one multi-output regressor (one model, five heads), rather than five independently trained networks.

Seven QoR experts. We train one such expert for each non-empty subset of $\{\mathrm{CDFG}, \mathrm{AST}, \mathrm{Text}\}$ (seven experts in total). All experts are frozen before KAS training, and the best-expert labels e^* follow the procedure in Section II-B.

D. Kernel-Aware Selector (KAS)

Fig. 5 illustrates the training and deployment pipeline of KAS. KAS selects one of seven pre-trained experts based on the multimodal kernel representation and the target QoR metric. The selector is trained via reinforcement learning, with rewards derived from validation prediction errors. We adopt reinforcement learning because expert selection must minimize downstream QoR prediction error at deployment, not merely fit best-expert labels e^*. PPO directly maximizes the performance ratio in Eq. (8), assigning higher reward to near-optimal choices and enabling exploration when several experts are similarly strong on validation—benefits that one-hot supervision on e^* cannot provide. We next detail its state representation, policy design, reward formulation, training procedure, and inference workflow.

State and policy. For a training or inference instance, the state s includes (i) a kernel-level context vector built from the cached embeddings in Sec. III-B and the MFF fusion stack in Sec. III-C, and (ii) an embedding of the QoR metric t. Metric-specific policy networks map s to logits over $\{0, \ldots, 6\}$, parameterizing $\pi_\theta(a \mid s)$.

Reward and PPO update. The policy is optimized with proximal policy optimization (PPO) [24]. During RL training, an action a selects expert e_a. Let $e_j^{(t)}(k,c)$ be the validation error of expert j on metric t. The immediate reward compares the chosen expert to the best available expert:

$$r(s,a) = \min_j \frac{e_j^{(t)}(k,c) + \epsilon}{e_a^{(t)}(k,c) + \epsilon}, \quad r \leftarrow \min(r, r_{\max}), \tag{8}$$

where ϵ avoids division by zero and $r_{\max}$ caps extreme ratios. Rewards are normalized within a mini-batch to form advantages. PPO maximizes a clipped surrogate objective with an entropy bonus to stabilize exploration.

Training. KAS policies are trained separately for each QoR metric using offline best-expert labels e^* from Sec. II-B. During PPO updates, actions closer to the e^* expert receive higher rewards (Eq. (8)). After convergence, one policy checkpoint is retained per QoR metric.

Inference. Given a new kernel and QoR metric t, the trained policy returns $\hat{e}(k,t) = \arg\max_a \pi_\theta(a \mid s)$. Only $f^{(t)}_{\mathcal{M}(\hat{e})}$ is executed to produce $\hat{y}^{(t)}$ (Eq. (4)). The online cost per metric is one policy forward pass plus one expert forward pass, rather than all seven regressors. Different metrics may select different experts for the same kernel, reflecting heterogeneous optimality of modality combinations across kernels and QoR dimensions.

End-to-end workflow. For an unseen design, the system first builds aligned CDFG, Text, and AST features as described in Section III-B. It then runs KAS once for each requested

TABLE II
TWENTY-TWO KERNELS FROM MACHSUITE AND POLYBENCH (†: SEVEN UNSEEN TEST KERNELS). THE TRAINING POOL CONTAINS 15 KERNELS WITH 4,355 DESIGN CONFIGURATIONS IN TOTAL; THE SEVEN † KERNELS ARE RESERVED FOR FINAL EVALUATION ONLY AND CONTAIN 1,935 DESIGN CONFIGURATIONS IN TOTAL. *Dataset Size* IS THE NUMBER OF DESIGN CONFIGURATIONS WITH GROUND TRUTH QOR LABELS.

Kernel	Description	Dataset Size
adi	Alternating direction implicit solver	322
aes	A common block cipher	45
atax†	Matrix transpose and vector multiplication	227
bicg	BiCG sub kernel of bicgstab linear solver	347
doitgen	multi-resolution analysis kernel	74
fdtd-2d	2-D finite different time domain kernel	289
gemm-blocked	A blocked version of matrix multiplication	243
gemm-ncubed	Dense matrix multiplication	362
gemver	Vector multiplication and matrix addition	464
gemm-p†	Matrix-multiply	303
gesummv†	Scalar, vector and matrix multiplication	159
mvt	matrix vector product and transpose	476
spmv-crs	Sparse matrix-vector multiplication(variable-length)	73
spmv-ellpack†	Sparse matrix-vector multiplication(fixed-size)	114
2mm	2 matrix multiplications	388
3mm	3 matrix multiplications	467
heat-3d	Heat equation over 3D data domain	225
jacobi-1d†	1-D Jacobi stencil computation	222
jacobi-2d†	2-D Jacobi stencil computation	524
nw†	A dynamic programming algorithm	386
seidel-2d	2-D Seidel stencil computation	56
stencil	A two-dimensional stencil computation	524

QoR metric and invokes only the recommended expert regressor. This design decouples multimodal QoR modeling via MFF from kernel aware expert deployment through KAS.

IV. EXPERIMENTAL EVALUATION

A. Experimental Setup

This section evaluates the QoR prediction accuracy of MFF-KAS using a consistent HLS experimental setup and fixed train/test data splitting strategy across all comparisons. We aim to answer three key questions:

1) Can integrating AST into multimodal representations significantly improve HLS QoR prediction over existing predictors and unimodal baselines?
2) How do different modality combinations perform across diverse kernels and QoR metrics, and what are their individual contributions?
3) Given that no single modality fusion is universally optimal, can the proposed kernel-aware selector (KAS) consistently improve prediction accuracy and generalization across unseen kernels?

a) Dataset: We use 22 kernels from MachSuite [18] and PolyBench [19], listed in Table II. Each design configuration uses three inputs: a ProGraML-based CDFG [13], a program AST, and tokenized source text [14]. Ground-truth QoR labels are obtained by synthesizing each kernels with its pragma settings on AMD UltraScale+ MPSoC ZCU104 using Vitis HLS and Vivado 2022.1.

b) Partition and metrics: We first fix a random seed and select seven kernels as unseen test benchmarks († in Table II): *atax*, *gemm-p*, *gesummv*, *jacobi-1d*, *jacobi-2d*, *nw*, and *spmv-ellpack*. The remaining 15 kernels contain 4,355 design configurations in total. These configurations are randomly split into training and validation sets with a ratio of 8:2. All regressors are trained on the training split and tuned on the validation split; KAS supervision labels are generated from the validation split (Sec. II-B). Final evaluation is conducted on all design configurations of the seven unseen kernels.

TABLE III
RMSE OF MPM, SOTA MODELS, AND MFF FUSION VARIANTS ON UNSEEN TEST KERNELS (MACRO-AVERAGED PER QOR METRIC). *All* IS THE SUM OF RMSE OVER LATENCY, LUT, DSP, FF, AND BRAM.

Method	Latency	LUT	DSP	FF	BRAM	All
GNN-DSE [4]	0.692	0.109	0.145	0.083	0.058	1.086
HGBO [5]	0.492	0.082	0.057	0.047	0.069	0.747
Ironman-Pro [6]	0.702	0.038	0.046	0.033	0.043	0.862
MPM [9]	0.459	**0.034**	**0.040**	**0.021**	0.034	0.588
AST-only	0.382	0.076	0.089	0.067	0.075	0.688
AST+CDFG	0.361	0.046	0.064	0.058	0.055	0.584
AST+Text	0.442	0.036	0.049	0.036	**0.032**	0.594
AST+CDFG+Text	**0.297**	0.067	0.058	0.042	0.047	**0.511**

We adopt root mean square error (RMSE) as the primary evaluation metric, as it is a standard choice in HLS QoR prediction and enables consistent error comparison across Latency, LUT, DSP, FF, and BRAM despite their different scales [3], [4]. For each QoR metric, we compute the RMSE on every unseen kernel and then report the unweighted mean over the seven kernels (macro average), ensuring that each test kernel contributes equally. Detailed per metric and per kernel results are presented in the following subsections.

c) Compared Baselines and Model Variants: Table III presents the RMSE results on unseen kernels for eight prediction methods. These include four state-of-the-art QoR predictors, namely GNN-DSE [4], HGBO [5], Ironman-Pro [6], and MPM [9], as well as four MFF variants incorporating AST features: AST-only, AST+CDFG, AST+Text, and AST+CDFG+Text. Table IV summarizes the full ablation study across all seven modality combinations: AST-only, CDFG-only, Text-only, and the three bimodal fusions and the trimodal fusion. MFF-KAS is evaluated separately. During inference, it performs kernel-level expert selection among the seven pre-trained fusion experts, with macro-averaged and per-kernel results reported in Tables VI and VII.

d) Implementation: Seven fusion experts (AST-only, CDFG-only, Text-only, AST+CDFG, AST+Text, CDFG+Text, and AST+CDFG+Text) are pretrained independently under the MFF architecture using the same data partition. Expert checkpoints are frozen before KAS training; at inference, only the selected expert is executed, and fusion weights are not retrained on the test set.The seven fusion experts and KAS share the same optimizer, learning rate, training epochs, and early-stopping rule on the validation set. SOTA QoR predictors presented in Table III (GNN-DSE, HGBO, Ironman-Pro, and MPM) follow their original implementations or published settings and are not required to use the same training configuration as the MFF experts.

TABLE IV
RMSE LOSS OF SEVEN FUSION EXPERTS ON UNSEEN KERNELS.

Fusion expert	Latency	LUT	DSP	FF	BRAM	All
AST-only	0.382	0.076	0.089	0.067	0.075	0.688
CDFG-only	0.418	0.081	0.086	0.064	0.095	0.744
Text-only	0.492	**0.035**	**0.046**	**0.031**	**0.031**	0.635
AST+CDFG	0.361	0.046	0.064	0.058	0.055	0.584
AST+Text	0.442	0.036	0.049	0.036	0.032	0.594
CDFG+Text	0.453	0.050	0.052	0.042	0.041	0.638
AST+CDFG+Text	**0.297**	0.067	0.058	0.042	0.047	**0.511**

B. Comparison with SOTA models

Table III reports macro-averaged RMSE on the seven unseen test kernels († in Table II). The upper four rows are prior learning-based QoR predictors; the lower four rows are MFF variants that incorporate AST. Among SOTA baselines, MPM [9] achieves the lowest *All* RMSE (0.588) and serves as our primary reference.

Adding AST consistently helps: AST-only (0.382 Latency) already beats all four baselines on Latency. AST+CDFG further improves Latency and *All* RMSE to 0.361 and 0.584; AST+Text is strongest on BRAM (0.032) but weaker on Latency than AST+CDFG. AST+CDFG+Text attains the best Latency (0.297) and *All* RMSE (0.511), i.e., 13.1% lower *All* than MPM and 25.7% lower than AST-only. Still, MPM and AST+Text win on several resource metrics, so even the best fixed trimodal fusion is not uniformly optimal across QoR metrics.

C. Evaluation of All Modality Subsets

Table IV reports RMSE for seven MFF experts trained as in Sec. IV-B on the seven unseen test kernels († in Table II).

AST-only attains the lowest unimodal Latency RMSE (0.382), ahead of *CDFG-only* (0.418) and *Text-only* (0.492). *Text-only* yields the best RMSE on all four resource metrics (LUT, DSP, FF, and BRAM). *CDFG-only* records the highest unimodal *All* RMSE (0.744).

Among bimodal experts, *AST+CDFG* achieves the second-best Latency (0.361) and a low *All* RMSE (0.584). *AST+Text* preserves strong BRAM accuracy (0.032) but degrades Latency to 0.442. *CDFG+Text* lowers *All* RMSE to 0.638 compared with *CDFG-only*, yet its Latency (0.453) remains higher than *CDFG-only*, and its resource RMSE is worse than *Text-only* on every utilization column.

AST+CDFG+Text achieves the best Latency (0.297) and best *All* RMSE (0.511), but it is not the per-column winner on resource metrics, where *Text-only* or *AST+Text* remain superior. This addresses research question (2) and further motivates kernel-aware selection.

D. No Single Fusion Expert Fits All Kernels and Metrics

The aforementioned results show that although *AST+CDFG+Text* achieves the best Latency (0.297) and the best overall RMSE (0.511) among the seven fusion strategies, it does not consistently outperform on every resource metric

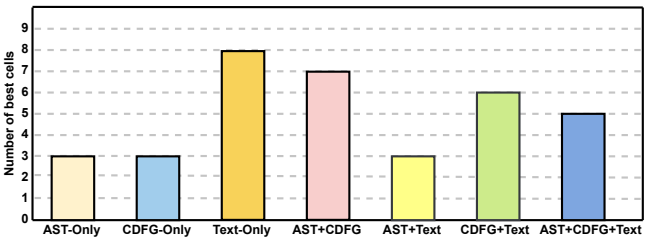


Fig. 6. Best fusion expert across unseen test kernels and QoR metrics. Each bar shows how many of the 35 (kernel, metric) pairs (seven kernels × five metrics) achieve the lowest RMSE with that expert.

TABLE V
MACRO-AVERAGED RMSE ON UNSEEN KERNELS: *AST+CDFG+Text* VS. MFF-KAS.

Method	Latency	LUT	DSP	FF	BRAM	All
AST+CDFG+Text	0.297	0.067	0.058	0.042	0.047	0.511
MFF-KAS	**0.223**	**0.033**	**0.040**	**0.028**	**0.031**	**0.355**

(LUT, DSP, FF, BRAM) for every kernel. Figure 6 further visualizes, for each unseen kernel and each QoR metric, which of the seven pretrained experts attains the lowest RMSE. The height of each bar indicates the number of winning (kernel, metric) pairs out of a total of 35. Notably, *Text-only* is optimal in eight cells, followed by *AST+CDFG* in seven and *CDFG+Text* in six, whereas *AST+CDFG+Text* is optimal in only five cells, despite having the best macro averaged Latency and overall RMSE in Tables III and IV. For instance, on the *nw* kernel, *AST+Text* yields the best Latency, while *Text-only* performs best on LUT and BRAM; on *spmv-ellpack*, *AST-only* achieves the best Latency but falls short on the four resource metrics. No single fusion expert dominates all 35 (kernel, metric) pairs, which further addresses research question (2).

E. Evaluation of Kernel-Aware Expert Selection

Given the heterogeneity observed in Fig. 6, we evaluate MFF-KAS, which dynamically routes each unseen test kernel to one of the pretrained fusion experts at inference time. We compare four strategies: (i) MPM: the best performing SOTA model. (ii) *AST+CDFG+Text*: the trimodal fusion expert applied uniformly to all unseen kernels. (iii) Best single expert per kernel: for each kernel, the expert with the lowest overall RMSE across seven fusion variants is selected and used for all five QoR metrics. (iv) MFF-KAS: our proposed method.

a) Macro-average results: Table V summarizes macro-averaged RMSE on the seven unseen test kernels. MFF-KAS reduces RMSE on every QoR metric relative to *AST+CDFG+Text*. *All* RMSE drops from 0.511 to 0.355 (30.5% relative reduction); Latency RMSE drops from 0.297 to 0.223 (24.9%). MFF-KAS also outperforms every fusion experts listed in Table IV on all five metrics.

b) Per-kernel results: Table VI compares per-kernel *All* RMSE on the seven unseen test kernels for MPM [9], fixed trimodal fusion (*AST+CDFG+Text*), the best single expert per kernel, and MFF-KAS. The Δ% column reports the relative

TABLE VI

PER-KERNEL *All* RMSE ON UNSEEN TEST KERNELS FOR MPM [9], FIXED TRIMODAL FUSION (*AST+CDFG+Text*), THE BEST SINGLE EXPERT PER KERNEL, AND MFF-KAS (LOWER IS BETTER). *All* IS THE SUM OF RMSE OVER LATENCY, LUT, DSP, FF, AND BRAM. $\Delta\%$: RELATIVE *All*-RMSE REDUCTION OF MFF-KAS OVER MPM [9].

Kernel	MPM [9]	AST+CDFG+Text	Best single expert	MFF-KAS	$\Delta\%$
atax	0.455	0.333	0.304	**0.303**	33.4
gemm-p	0.881	0.496	**0.444**	0.451	48.8
gesummv	0.619	0.354	0.354	**0.316**	49.0
jacobi-1d	0.748	0.314	0.314	**0.285**	61.9
jacobi-2d	0.628	0.485	0.416	**0.337**	46.3
nw	0.284	0.583	0.289	**0.252**	11.3
spmv-ellpack	0.759	0.961	0.757	**0.460**	39.4
Macro average	0.588	0.511	0.418	**0.355**	39.6

All-RMSE reduction of MFF-KAS over MPM (Table caption). Compared with MPM, MFF-KAS lowers the macro-averaged *All* RMSE from 0.588 to 0.355 ($\Delta\%$ = 39.6%) and improves every kernel, with per-kernel gains ranging from 11.3% (*nw*) to 61.9% (*jacobi-1d*). The largest reductions appear on *jacobi-1d*, *gemm-p*, *gesummv*, and *spmv-ellpack*. Relative to fixed trimodal fusion, MFF-KAS also reduces macro-averaged *All* RMSE from 0.511 to 0.355 (30.5%). Together, these results show that neither a single fixed SOTA predictor nor a uniform trimodal fusion generalizes well on unseen kernels, motivating kernel- and metric-aware expert routing via KAS.

F. Discussion and Summary

Our experiments yield three key findings aligned with the research questions. First, integrating AST into multimodal representations significantly outperforms state-of-the-art predictors and unimodal/bimodal variants. The AST+CDFG+Text fusion achieves the lowest macro-averaged RMSE for latency and overall QoR on unseen kernels (Tables III and IV). Second, the optimal modality combination varies across kernels and metrics (Fig. 6). Text-only performs best in eight of 35 kernel-metric pairs, while AST+CDFG+Text is optimal in only five, confirming no single fusion strategy is universally superior. Third, MFF-KAS reduces the macro-averaged overall RMSE from 0.511 to 0.355 (30.5%) over the fixed AST+CDFG+Text baseline, with consistent improvements across all seven unseen kernels (Tables V and VI).

V. CONCLUSION

This paper proposed MFF-KAS for HLS QoR prediction. We introduced AST as a syntactic representation to complement CDFG and source text, capturing pragma-loop structures that existing methods miss. Systematically evaluating all seven modality subsets revealed that no single fixed fusion is optimal across kernels and QoR metrics. We therefore developed a kernel-aware selector that dynamically chooses the best pretrained expert for each unseen kernel and QoR metric. On unseen benchmarks, MFF-KAS reduces macro-averaged RMSE by 30.5% compared to the best trimodal fusion and outperforms strong baselines, demonstrating effective cross-kernel generalization. Future work will extend selection from kernel level to individual design configurations.

REFERENCES

[1] A. Sohrabizadeh, C. H. Yu, M. Gao, and J. Cong, "AutoDSE: Enabling software programmers to design efficient FPGA accelerators," *ACM Trans. Design Autom. Electron. Syst.*, vol. 27, no. 4, Art. 32, Feb. 2022, doi: 10.1145/3494534.

[2] B. Carrion-Schafer and Z. Wang, "High-level synthesis design space exploration: Past, present, and future," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 10, pp. 2628–2639, Oct. 2020.

[3] S. Dai, Y. Zhou, H. Zhang, E. Ustun, E. F. Y. Young, and Z. Zhang, "Fast and accurate estimation of quality of results in high-level synthesis with machine learning," in *Proc. IEEE Symp. Field-Program. Custom Comput. Mach. (FCCM)*, 2018, pp. 129–132, doi: 10.1109/FCCM.2018.00029.

[4] A. Sohrabizadeh, Y. Bai, Y. Sun, and J. Cong, "Automated accelerator optimization aided by graph neural networks," in *Proc. ACM/IEEE Design Autom. Conf. (DAC)*, 2022, pp. 55–60.

[5] H. Kuang, X. Cao, J. Li, and L. Wang, "HGBO-DSE: Hierarchical GNN and Bayesian optimization based HLS design space exploration," in *Proc. Int. Conf. Field Program. Technol. (ICFPT)*, 2023, pp. 106–114.

[6] N. Wu, Y. Xie, and C. Hao, "IronMan-Pro: Multi-objective design space exploration in HLS via reinforcement learning and graph neural network based modeling," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 3, pp. 900–913, Mar. 2023.

[7] Z. Wang and B. Carrion-Schafer, "Learning from the Past: Efficient High-Level Synthesis Design Space Exploration for FPGAs," *ACM Trans. Design Autom. Electron. Syst.*, vol. 27, no. 4, Art. 34, Feb. 2022, doi: 10.1145/3495531.

[8] Z. Qin *et al.*, "Cross-modality program representation learning for electronic design automation with high-level synthesis," in *Proc. ACM/IEEE Int. Symp. Mach. Learn. CAD (MLCAD)*, 2024, pp. 1–12.

[9] L. Xu *et al.*, "MPM-LLM4DSE: Reaching the Pareto frontier in HLS with multimodal learning and LLM-driven exploration," in *Proc. IEEE Design, Automation and Test in Europe Conference (DATE)*, 2026, pp. 1–7.

[10] M. Z. S. Khan, N. Mashnoor, M. Akyash, K. Azar, and H. Kamali, "SAGE-HLS: Syntax-aware AST-guided LLM for high-level synthesis code generation," in *Proc. IEEE Int. Conf. Comput. Design (ICCD)*, 2025, pp. 574–581.

[11] D. H. Wolpert and W. G. Macready, "No free lunch theorems for optimization," *IEEE Trans. Evol. Comput.*, vol. 1, no. 1, pp. 67–82, Apr. 1997, doi: 10.1109/4235.585893.

[12] L. Kotthoff, "Algorithm selection for combinatorial search problems: A survey," *AI Mag.*, vol. 35, no. 3, pp. 48–60, Fall 2014.

[13] C. Cummins *et al.*, "ProGraML: Graph-based deep learning for program optimization and analysis," in *Proc. MLSys*, 2021, vol. 3, pp. 548–562.

[14] D. Feng *et al.*, "CodeBERT: A pre-trained model for programming and natural languages," in *Proc. EMNLP*, 2020, pp. 1536–1547.

[15] D. Guo *et al.*, "GraphCodeBERT: Pre-training code representations with data flow," in *Proc. ICLR*, 2021, arXiv:2009.08366.

[16] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. ICLR*, 2017.

[17] A. Vaswani *et al.*, "Attention is all you need," in *Proc. NeurIPS*, 2017, pp. 5998–6008.

[18] B. Reagen *et al.*, "MachSuite: Benchmarks for accelerator design and customized architectures on FPGA," *IEEE Micro*, vol. 34, no. 3, pp. 18–26, May–Jun. 2014.

[19] M. Á. Abella-González, L.-N. Pouchet, and G. Rodríguez, "PolyBench/Python: Benchmarking Python environments with polyhedral optimizations," in *Proc. ACM SIGPLAN Int. Conf. Compiler Construction (CC)*, 2021, pp. 59–70.

[20] S. Pouget, L.-N. Pouchet, and J. Cong, "Automatic hardware pragma insertion in high-level synthesis: A non-linear programming approach," *ACM Trans. Design Autom. Electron. Syst.*, vol. 30, no. 2, Art. 26, Feb. 2025.

[21] R. Nane *et al.*, "A survey and evaluation of FPGA high-level synthesis tools," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 35, no. 10, pp. 1591–1604, Oct. 2016.

[22] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis and transformation," in *Proc. CGO*, 2004, pp. 75–88.

[23] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *Proc. ICLR*, 2019.

[24] J. Schulman *et al.*, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.